

CONFIDENTIEL



Version : 1.0 | **Date :** 2 juin 2026

Classification : Confidentiel — Usage interne Thymus

Destinataires : Equipe d'ingénierie Thymus

THYMUS

Document de Conception Technique et Philosophique

Version : 1.0

Date : 2 juin 2026

Classification : Confidentiel - Usage interne Thymus

Destinataires : Equipe d'ingénierie Thymus

TABLE DES MATIERES

1. Résumé exécutif
 2. Le problème fondamental
 3. L'inspiration biologique
 4. La philosophie Thymus
 5. Les 6 mécanismes immunitaires
 6. Architecture globale
 7. Composants techniques
 8. Modèles de données
 9. Algorithmes fondamentaux
 10. Cartographie vivante
 11. Système immunitaire collectif
 12. Démonstrations
 13. Interface et expérience utilisateur
 14. Roadmap technique
 15. Equipe et organisation
 16. Positionnement stratégique
 17. Annexes techniques
-

1. RESUME EXECUTIF

Thymus est une plateforme de cybersécurité fondée sur une philosophie inédite : traiter un réseau informatique comme un organisme vivant doté d'un système immunitaire complet.

Les solutions de cybersécurité actuelles (EDR, NDR, XDR, SIEM) fonctionnent selon un paradigme commun : elles cherchent des attaques. Elles comparent ce qu'elles observent à des bases de signatures, des indicateurs de compromission, ou des modèles statistiques d'anomalies.

Thymus inverse ce paradigme.

Thymus ne cherche pas les attaques. Thymus construit une identité complète du réseau -- son "Soi" -- et réagit à tout ce qui ne correspond pas à cette identité. Exactement comme le système immunitaire humain ne connaît pas toutes les maladies possibles, mais connaît intimement les cellules du corps et attaque tout ce qui n'en fait pas partie.

Cette approche résout un problème fondamental : comment détecter une attaque qui n'a jamais été vue, qui n'utilise aucun malware connu, qui emploie des identifiants légitimes et des outils système normaux ?

La réponse : en détectant non pas l'attaque elle-même, mais la mutation qu'elle provoque dans l'identité du réseau.

2. LE PROBLEME FONDAMENTAL

2.1 L'évolution des cyberattaques

Les cyberattaques de 2026 ne ressemblent plus à celles de 2015.

Avant : un malware avec une signature identifiable pénètre le réseau, tente d'exécuter du code malveillant, déclenche un antivirus.

Aujourd'hui : un attaquant obtient des identifiants légitimes (phishing, achat sur le dark web, ingénierie sociale), se connecte normalement, utilise des outils système standards (PowerShell, SSH, RDP), se déplace latéralement dans le réseau, et exfiltre des données via des canaux chiffrés autorisés.

Dans ce scénario :

- Les identifiants sont valides
- Les connexions sont autorisées
- Les outils utilisés sont légitimes
- Le trafic est chiffré
- Aucun malware n'est déployé

Les systèmes de sécurité classiques sont aveugles.

2.2 Les limites des approches actuelles

EDR (Endpoint Detection and Response)

Vision : la machine individuelle.

Force : détection des malwares, analyse des processus.

Faiblesse : ne comprend pas les relations entre machines. Quand l'attaquant utilise des outils légitimes, l'EDR ne voit rien d'anormal sur l'endpoint.

NDR (Network Detection and Response)

Vision : le trafic réseau.

Force : analyse des flux, détection des communications suspectes.

Faiblesse : traite le réseau comme un tuyau, pas comme un organisme. Détecte des anomalies de trafic mais ne comprend pas la logique métier des relations.

SIEM (Security Information and Event Management)

Vision : les logs centralisés.

Force : corrélation d'événements, conformité.

Faiblesse : réactif, dépendant de règles écrites par des humains, noyé sous les faux positifs, nécessite une équipe SOC dédiée.

XDR (Extended Detection and Response)

Vision : corrélation multi-sources.

Force : vue plus large que l'EDR seul.

Faiblesse : reste fondamentalement un système qui cherche des patterns d'attaque connus. L'approche est toujours "chercher le mal".

2.3 Le vide philosophique

Toutes ces solutions partagent le même paradigme :

"Nous connaissons les attaques. Nous les cherchons."

Ce paradigme échoue face à :

- Les attaques zero-day (aucune signature connue)
- Les attaques sans malware (living-off-the-land)
- Les menaces internes (utilisateurs légitimes malveillants)
- Les attaques lentes (exfiltration sur plusieurs semaines)
- Les attaques sur mesure (APT ciblant une organisation spécifique)

Thymus propose un paradigme inverse :

"Nous connaissons le réseau. Nous détectons tout ce qui ne lui ressemble pas."

3. L'INSPIRATION BIOLOGIQUE

3.1 Le système immunitaire humain

Le corps humain fait face à des millions de pathogènes potentiels : virus, bactéries, parasites, cellules cancéreuses. Beaucoup n'ont jamais été rencontrés auparavant.

Pourtant, le système immunitaire parvient à protéger l'organisme sans connaître à l'avance toutes les menaces possibles.

Comment ?

En ne cherchant pas les maladies, mais en connaissant intimement le "Soi" -- l'ensemble des cellules et molécules qui appartiennent au corps -- et en attaquant systématiquement tout ce qui est "Non-Soi".

3.2 Les mécanismes clés

Le système immunitaire humain possède plusieurs mécanismes que Thymus reproduit :

Immunité innée

Défenses présentes dès la naissance : la peau, les muqueuses, les cellules tueuses naturelles (NK). Ces défenses sont génériques et immédiates. Elles ne nécessitent aucun apprentissage.

Immunité adaptative

Défenses développées au contact des menaces : les lymphocytes B (anticorps) et les lymphocytes T. Ces défenses sont spécifiques à chaque pathogène et deviennent plus efficaces avec le temps.

Distinction Soi / Non-Soi

Le thymus forme les lymphocytes T à reconnaître les cellules du corps. Les lymphocytes qui attaqueraient le corps sont éliminés. Ceux qui reconnaissent correctement le Soi survivent.

Lymphocytes T mémoire

Après avoir combattu une infection, le système immunitaire conserve des cellules mémoire pendant des années. Lors d'une réexposition, la réponse est beaucoup plus rapide.

Réponse inflammatoire locale

La première réponse à une agression est locale : rougeur, chaleur, gonflement au site de l'infection. Le corps ne mobilise pas l'ensemble de ses défenses pour une coupure au doigt.

Sélection clonale

Quand un anticorps efficace est trouvé, il est massivement cloné. Les anticorps inefficaces sont naturellement éliminés. Le système s'auto-optimise.

Tolérance immunitaire

Le système apprend à ne PAS réagir aux éléments inoffensifs : nourriture, flore intestinale, cellules du corps. Sans ce mécanisme, le corps s'attaquerait lui-même (maladies auto-immunes).

3.3 Tableau de correspondance Biologie / Thymus

Biologie	Thymus	Fonction
Cellule du corps	Machine/service du réseau	Composant légitime
Pathogène	Attaque, intrusion, exfiltration	Menace
Anticorps	Règle de détection adaptative	Défense spécifique
Peau / Muqueuses	Firewall, segmentation, règles innées	Défense générique
Thymus	Phase d'apprentissage du Soi	Formation des détecteurs
Lymphocyte mémoire	T Cellule mémoire Thymus	Réponse accélérée
Réponse inflammatoire	Réponse locale du sensor	Confinement immédiat
Sélection clonale	Amplification des règles efficaces	Auto-optimisation
Tolérance	Apprentissage des comportements rares légitimes	Réduction des faux positifs
Système lymphatique	Réseau de communication entre sensors	Propagation de l'intelligence
Fièvre	Réponse systémique : mode alerte réseau	Mobilisation générale
ADN cellulaire	ADN comportemental de la machine	Identité

4. LA PHILOSOPHIE Thymus

4.1 Le réseau comme organisme

Thymus ne considère pas le réseau comme une infrastructure technique composée de machines, de câbles et de protocoles.

Thymus considère le réseau comme un organisme vivant composé de :

- **Cellules** : les machines, les services, les utilisateurs
- **Organes** : les groupes fonctionnels (département RH, finance, production)
- **Flux sanguins** : les communications réseau
- **Système nerveux** : les protocoles de contrôle et d'administration
- **Métabolisme** : les volumes de données échangés

Cet organisme a une identité : son ADN comportemental.

4.2 L'ADN comportemental

Chaque machine du réseau possède un ADN en trois dimensions.

ADN technique

Ce que la machine EST :

- Système d'exploitation et version
- Applications installées
- Services en écoute
- Ports ouverts
- Ressources matérielles (CPU, RAM, disque)

ADN relationnel

Avec QUI la machine communique :

- Liste des machines contactées habituellement
- Fréquence de chaque relation

- Sens des communications (initiateur ou répondeur)
- Protocoles utilisés dans chaque relation
- Volume échangé avec chaque pair

ADN temporel

QUAND et COMBIEN la machine communique :

- Plages horaires d'activité
- Jours de la semaine vs week-end
- Pics d'activité récurrents
- Volume moyen par heure, par jour, par semaine
- Saisonnalité (fins de mois, périodes fiscales)

4.3 Le concept de mutation

Dans un organisme sain, les cellules se comportent conformément à leur ADN. Quand une cellule commence à se comporter de manière radicalement différente -- par exemple en se multipliant de façon incontrôlée -- c'est potentiellement un cancer.

De même, quand une machine du réseau commence à agir en contradiction avec son ADN comportemental, c'est potentiellement une compromission.

Thymus ne détecte pas des attaques. Thymus détecte des mutations.

Une mutation est un changement dans l'ADN comportemental d'une machine ou d'un groupe de machines qui dépasse les variations normales observées.

Exemples de mutations :

Mutation observée	Interprétation possible
Machine RH contacte le serveur Trésor	Relation nouvelle, jamais observée
Volume sortant passe de 300 Mo à 50 Go	Mutation volumétrique massive
Activité à 3h du matin sur un poste bureau	Mutation temporelle
Poste utilisateur scanne 15 machines en 2 min	Mutation relationnelle explosive
Service HTTP apparaît sur un serveur de fichiers	Mutation technique

5. LES 6 MECANISMES IMMUNITAIRES

5.1 Mécanisme 1 : Distinction Soi / Non-Soi

Principe biologique

Le thymus forme les lymphocytes T en les exposant aux protéines du corps (CMH). Seuls les lymphocytes capables de reconnaître le Soi sans l'attaquer survivent. Ce processus s'appelle la sélection thymique.

Le résultat : le système immunitaire possède un modèle complet de ce qui est "normal" dans le corps, et réagit à tout ce qui ne correspond pas.

Implémentation Thymus

Thymus implémente une phase d'apprentissage appelée **"Phase Thymus"**.

Pendant cette phase (2 à 4 semaines minimum), Thymus observe le réseau sans intervenir. Il construit le modèle du Soi :

```
Phase Thymus
=====

Durée : 14-28 jours (configurable)
Mode : observation pure, aucune alerte, aucune action

Le système construit :
```

1. Inventaire complet des machines
2. Cartographie des relations normales
3. Profils volumétriques de chaque machine
4. Profils temporels de chaque machine
5. Profils des protocoles et ports utilisés
6. Identification des groupes fonctionnels
7. Baseline du métabolisme réseau global

A la fin de la Phase Thymus, Thymus possède un modèle du Soi -- l'état normal du réseau dans toutes ses dimensions.

Différence fondamentale avec les EDR/NDR : les systèmes classiques construisent des baselines statistiques (moyennes, écarts-types). Thymus construit un modèle d'identité multidimensionnel. Ce n'est pas "cette machine transfère en moyenne 300 Mo". C'est "cette

machine est une cellule de type RH, qui communique avec ces 4 pairs, via ces protocoles, à ces horaires, avec ces volumes, et qui fait partie de l'organe Ressources Humaines". Le contexte est entier.

Architecture du Soi

```
SelfModel {
  machines: [MachineIdentity], // toutes les cellules
  relations: [RelationProfile], // toutes les connexions normales
  organs: [FunctionalGroup], // groupes fonctionnels
  metabolism: NetworkMetabolism, // métabolisme global
  rhythms: TemporalRhythms, // rythmes circadiens du réseau
  confidence: f64, // maturité du modèle (0.0 -> 1.0)
}
```

Le modèle du Soi n'est jamais figé. Il évolue lentement au fil du temps pour intégrer les changements légitimes (nouveau collaborateur, nouveau service, migration). Mais les changements brusques -- les mutations -- sont détectés.

Indice de confiance du Soi

Le modèle du Soi possède un indice de confiance qui augmente avec le temps :

```
Jour 1-7   : confiance 0.3 (modèle fragile, seuils larges)
Jour 7-14  : confiance 0.5 (modèle stabilisé, seuils moyens)
Jour 14-28 : confiance 0.7 (modèle fiable, seuils normaux)
Jour 28+   : confiance 0.8+ (modèle mature, seuils précis)
```

Les seuils de détection s'affinent automatiquement avec la maturité du modèle. Un modèle jeune tolère plus de variations. Un modèle mature est plus sensible.

5.2 Mécanisme 2 : Lymphocytes T mémoire

Principe biologique

Après une infection, le système immunitaire ne revient pas à zéro. Il conserve des lymphocytes T mémoire qui peuvent persister des décennies. Lors d'une réexposition au même pathogène, la réponse est 10 à 100 fois plus rapide.

C'est le principe de la vaccination : exposer le corps à une version affaiblie du pathogène pour créer des cellules mémoire sans subir la maladie.

Implémentation Thymus

Thymus maintient une **Mémoire Immunitaire** locale à chaque déploiement.

Chaque incident détecté et confirmé est transformé en une **Cellule Mémoire** :

```
MemoryCell {
  id: uuid,
  created_at: timestamp,

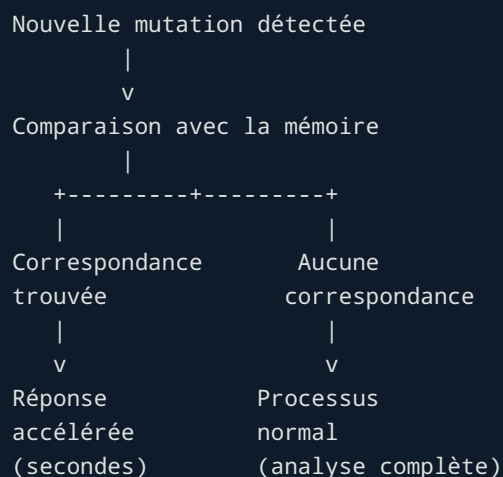
  // La mutation qui a déclenché l'alerte
  mutation_pattern: MutationSignature {
    affected_dimensions: [Technique | Relationnel | Temporel],
    deviation_profile: DeviationVector,
    progression_sequence: [MutationStep],
  },

  // Le contexte de l'incident
  context: IncidentContext {
    origin_machine: MachineId,
    affected_machines: [MachineId],
    attack_path: [PathSegment],
    duration: Duration,
    data_at_risk: DataEstimate,
  },

  // La réponse qui a fonctionné
  effective_response: ResponseSequence {
    actions_taken: [Action],
    time_to_containment: Duration,
    success_score: f64,
  },

  // Métadonnées d'efficacité
  times_matched: u32,          // combien de fois cette mémoire a servi
  false_positive_count: u32,    // combien de faux positifs
  last_matched: timestamp,
  effectiveness: f64,           // score d'efficacité (0.0 -> 1.0)
}
```

Quand Thymus détecte une nouvelle mutation, il la compare d'abord à sa mémoire immunitaire :



La différence avec une base de signatures : une signature est exacte (hash MD5, pattern binaire). Une Cellule Mémoire est un profil de mutation multidimensionnel. Elle reconnaît des attaques similaires mais pas identiques -- comme le système immunitaire reconnaît une variante de la grippe même s'il n'a jamais vu cette souche exacte.

Mémoire de vaccination

Thymus peut être "vacciné" : on peut injecter des Cellules Mémoire issues d'incidents survenus dans d'autres déploiements Thymus (via le système immunitaire collectif, voir section 11) ou créées manuellement à partir de rapports de menaces publics (CERT, ANSSI, etc.).

Le réseau acquiert une immunité contre des attaques qu'il n'a jamais subies.

5.3 Mécanisme 3 : Tolérance immunitaire

Principe biologique

Le système immunitaire doit être capable de ne PAS attaquer les cellules du corps, les aliments digérés, la flore intestinale. Cette capacité s'appelle la tolérance immunitaire.

Quand ce mécanisme échoue, le corps s'attaque lui-même : c'est une maladie auto-immune (lupus, sclérose en plaques, diabète de type 1). Ces maladies sont aussi dangereuses que les infections.

Le problème en cybersécurité

L'équivalent en cybersécurité, c'est le **faux positif** -- une alerte générée pour un comportement légitime. Les faux positifs sont le problème numéro 1 de tous les systèmes de détection comportementale.

Un système qui génère trop de faux positifs est inutilisable :

- Les équipes ignorent les alertes (alert fatigue)
- Les vrais incidents sont noyés dans le bruit
- Le système perd sa crédibilité
- Les utilisateurs le désactivent

C'est la maladie auto-immune de la cybersécurité : le système de défense attaque l'organisme qu'il est censé protéger.

Implémentation Thymus

Thymus implémente trois mécanismes de tolérance :

Tolérance centrale (pendant la Phase Thymus)

Pendant l'apprentissage initial, Thymus identifie les comportements rares mais légitimes :

Exemples de comportements rares légitimes :

- Backup hebdomadaire le dimanche à 2h (rare mais régulier)
- Mise à jour trimestrielle de l'ERP (pic massif de trafic, 4 fois par an)
- Audit externe annuel (machines inconnues sur le réseau pendant 2 semaines)
- Fin de mois comptable (volume x10 pendant 3 jours)

Ces comportements sont intégrés au modèle du Soi avec un tag "rare mais connu" :

```
ToleranceEntry {
  pattern: BehaviorPattern,
  frequency: Frequency,      // quotidien, hebdomadaire, mensuel, annuel
  last_seen: timestamp,
  expected_next: timestamp,
  confidence: f64,
  source: Manual | Learned,  // déclaré par l'admin ou appris automatiquement
}
```


Tolérance périphérique (en continu)

Quand Thymus génère une alerte et que l'administrateur la marque comme faux positif, Thymus apprend :

```
Alerte générée → Admin marque "Faux Positif"
  |
  v
Thymus crée une ToleranceEntry
  |
  v
Le même comportement ne déclenche plus d'alerte
(or déclenche une alerte de moindre sévérité)
```

Mais la tolérance n'est jamais absolue. Si le comportement toléré change de nature (même source mais volume 100x supérieur, ou même type mais à un horaire radicalement différent), Thymus réactive l'alerte.

Tolérance de contexte

Certains événements créent un contexte temporaire qui modifie la tolérance :

```
Contextes reconnus :

- Mise à jour système en cours → tolérer les redémarrages de services
- Migration annoncée → tolérer les nouvelles connexions entre machines
- Période de clôture comptable → tolérer les volumes élevés
- Maintenance planifiée → tolérer l'accès admin inhabituel
```

L'administrateur peut déclarer un contexte. Thymus ajuste automatiquement ses seuils pendant la durée du contexte, puis revient au mode normal.

```
POST /api/context
{
  "type": "planned_maintenance",
  "affected_machines": ["srv-db-01", "srv-app-01"],
  "start": "2026-06-15T22:00:00",
  "end": "2026-06-16T06:00:00",
  "tolerance_adjustments": {
    "new_connections": "tolerate",
    "service_restarts": "tolerate",
    "volume_spike": "tolerate_up_to_10x"
  }
}
```

5.4 Mécanisme 4 : Réponse inflammatoire locale

Principe biologique

Quand une bactérie pénètre par une coupure, la première réponse est locale : les cellules du site libèrent des médiateurs chimiques, les vaisseaux sanguins se dilatent (rougeur), du fluide s'accumule (gonflement), la température locale augmente. Cette inflammation contient l'infection localement avant que le système immunitaire adaptatif ne se mobilise.

Le corps ne mobilise pas l'ensemble de ses défenses pour une coupure au doigt. La réponse est proportionnelle et locale.

Le problème en cybersécurité

Les architectures de sécurité actuelles sont centralisées : les agents envoient des données à un serveur central qui prend les décisions. Si le serveur central est injoignable (réseau saturé, attaque DDoS, panne), les agents sont aveugles.

De plus, le temps de réponse dépend du temps de communication avec le serveur central. Dans une attaque rapide (ransomware qui chiffre en minutes), ce délai peut être fatal.

Implémentation Thymus

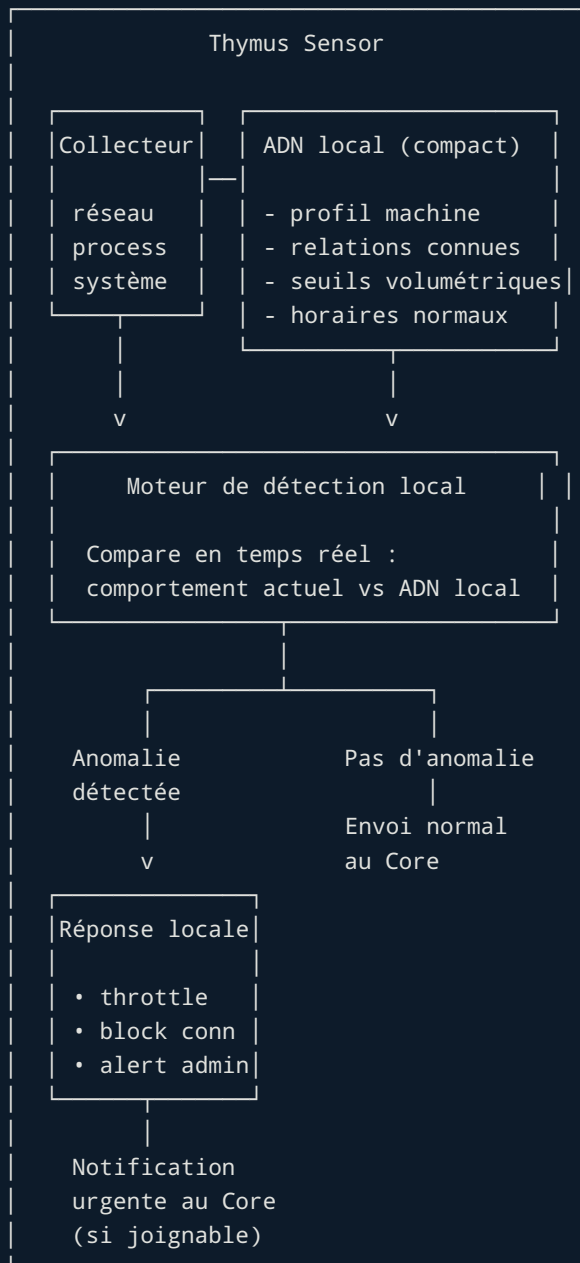
Chaque Thymus Sensor (l'agent installé sur les machines) possède une capacité de réponse locale autonome.

Le sensor ne se contente pas de collecter et d'envoyer des données. Il embarque :

1. Une copie compacte du profil de sa machine (son ADN)
2. Un moteur de détection local simplifié
3. Une capacité de réponse immédiate

Architecture du Sensor

=====



Niveaux de réponse locale

Le sensor peut appliquer 4 niveaux de réponse sans attendre le Core :

Niveau 1 : SURVEILLANCE

Condition : score de risque > 0.4
 Action : fréquence de collecte x5, logs détaillés
 Impact : aucun pour l'utilisateur
 Durée : jusqu'à validation par le Core

Niveau 2 : RALENTISSEMENT

Condition : score de risque > 0.6
 Action : limitation du débit réseau sortant
 Impact : transferts plus lents
 Durée : jusqu'à décision du Core (max 30 min)

Niveau 3 : BLOCAGE SELECTIF

Condition : score de risque > 0.8
 Action : blocage des connexions vers les destinations inconnues
 Impact : seules les connexions habituelles restent ouvertes
 Durée : jusqu'à décision du Core (max 15 min)

Niveau 4 : ISOLATION

Condition : score de risque > 0.95 ou détection d'exfiltration active
 Action : coupure de toutes les connexions sauf vers le Core
 Impact : machine isolée du réseau
 Durée : jusqu'à intervention manuelle

Indépendance du Core

Si le Core est injoignable (panne, attaque sur le Core lui-même, réseau saturé), le sensor continue de fonctionner :

- Il détecte localement
- Il répond localement
- Il stocke les événements dans un buffer local
- Quand le Core redevient joignable, il synchronise tout

Cette autonomie est fondamentale. Dans un vrai incident, la connectivité au serveur central est souvent la première chose qui tombe.

5.5 Mécanisme 5 : Sélection clonale

Principe biologique

Quand un anticorps reconnaît un pathogène, le lymphocyte B qui le produit est massivement cloné (expansion clonale). L'organisme produit des millions de copies de l'anticorps efficace.

En parallèle, les lymphocytes B qui ne reconnaissent rien ou qui reconnaissent le Soi (auto-réactifs) sont éliminés par apoptose (mort cellulaire programmée).

Résultat : le système immunitaire s'auto-optimise. Les bonnes défenses prolifèrent, les mauvaises disparaissent.

Implémentation Thymus

Thymus implémente une boucle d'auto-optimisation pour ses règles de détection.

Chaque règle de détection (innée ou adaptative) possède un score d'efficacité :

```
DetectionRule {
  id: uuid,
  rule_type: Innate | Adaptive,

  // Ce que la règle détecte
  pattern: MutationPattern,
  threshold: f64,

  // Statistiques d'efficacité
  stats: RuleStats {
    true_positives: u32, // alertes confirmées comme vraies menaces
    false_positives: u32, // alertes marquées comme faux positifs
    true_negatives: u32, // pas d'alerte, pas d'incident
    false_negatives: u32, // pas d'alerte, mais incident réel

    effectiveness: f64, // calculé automatiquement
    last_evaluated: timestamp,
  },

  // Statut évolutif
  status: Active | Amplified | Attenuated | Eliminated,
}
```

Boucle d'évolution

Chaque semaine, Thymus évalue ses règles :

$$\text{effectiveness} = \text{true_positives} / (\text{true_positives} + \text{false_positives} + \text{false_negatives})$$

Si $\text{effectiveness} > 0.8$:

- AMPLIFIER : baisser le seuil de déclenchement (plus sensible)
- Propager la règle aux autres sensors

Si effectiveness entre 0.4 et 0.8 :

- MAINTENIR : pas de changement

Si effectiveness entre 0.2 et 0.4 :

- ATTENUER : augmenter le seuil (moins sensible)
- Réduire la sévérité des alertes

Si $\text{effectiveness} < 0.2$:

- ELIMINER : désactiver la règle
- Archiver pour analyse

Conséquence

Le système devient plus précis avec le temps sans intervention humaine. Les règles qui fonctionnent deviennent plus sensibles. Les règles qui génèrent du bruit sont atténuées puis éliminées.

Après 6 mois de fonctionnement, le Thymus d'une organisation est un système de défense unique, optimisé spécifiquement pour ce réseau, que aucun attaquant ne peut anticiper car il est le produit de l'environnement local.

5.6 Mécanisme 6 : Immunité innée vs Immunité adaptative

Principe biologique

Le système immunitaire humain possède deux couches complémentaires :

Immunité innée : présente dès la naissance, identique chez tous les humains, réponse immédiate (minutes), générique. Comprend : la peau, les muqueuses, les cellules NK, le complément, la phagocytose.

Immunité adaptative : développée au contact des pathogènes, unique à chaque individu, réponse différée (jours), spécifique. Comprend : les lymphocytes B (anticorps), les lymphocytes T (destruction des cellules infectées).

Les deux couches sont indispensables. L'immunité innée contient l'infection immédiatement. L'immunité adaptative la neutralise définitivement.

Implémentation Thymus

Couche innée (présente dès l'installation)

Thymus est livré avec un ensemble de règles universelles qui fonctionnent sans apprentissage :

Règles innées (exemples) :

RESEAU

- Scan de ports (> 10 ports en < 1 min sur une même cible)
- Communication avec IP blacklistées (listes publiques de C2 connus)
- Protocoles DNS anormaux (tunneling DNS : requêtes > 512 octets fréquentes)
- ARP spoofing (changement MAC/IP non autorisé)

PROCESSUS

- Exécution depuis /tmp ou répertoires temporaires
- Processus tentant de désactiver les logs
- Elévation de privilèges non autorisée
- Accès en masse à des fichiers sensibles (> 100 fichiers en < 1 min)

SYSTEME

- Modification des fichiers de configuration système
- Création de comptes utilisateur non autorisée
- Modification des tâches planifiées (cron/scheduled tasks)
- Chargement de modules noyau inconnus

Ces règles sont identiques pour tous les déploiements Thymus. Elles sont maintenues et mises à jour par l'équipe Thymus.

Couche adaptative (développée par l'apprentissage)

Après la Phase Thymus, Thymus développe des règles spécifiques à ce réseau :

Règles adaptatives (exemples pour un réseau spécifique) :

- La machine rh-01 ne communique jamais avec le sous-réseau 10.0.3.x
→ Si elle le fait : score de risque +0.7
- Le serveur db-finance accepte des connexions uniquement de app-finance-01 et app-finance-02
→ Toute autre source : score de risque +0.8
- Le volume sortant vers Internet ne dépasse jamais 5 Go/jour
→ Au-delà : score de risque proportionnel
- Aucune machine du réseau interne n'utilise le port 4444
→ Connexion sur ce port : score de risque +0.9

Ces règles sont uniques à chaque déploiement. Elles évoluent en permanence avec le réseau.

Interaction des deux couches



Si la couche innée détecte quelque chose (scan de ports, C2 connu), la réponse est immédiate et forte. La couche adaptative n'est même pas consultée.

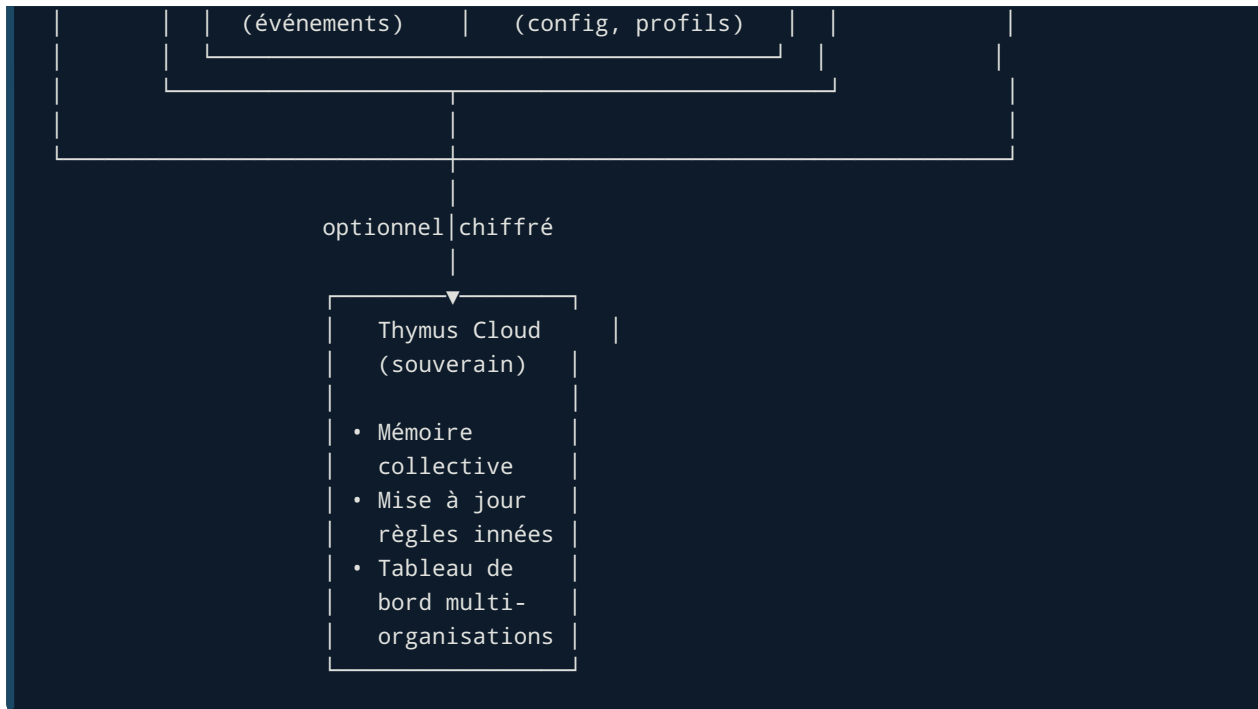
Si la couche innée ne détecte rien, la couche adaptative analyse l'événement dans le contexte spécifique du réseau.

Les deux scores se combinent : un événement peut être modérément suspect du point de vue inné (protocole inhabituel) ET modérément suspect du point de vue adaptatif (machine qui ne communique jamais avec cette destination), ce qui donne un score combiné élevé.

6. ARCHITECTURE GLOBALE

6.1 Vue d'ensemble





6.2 Flux de données

1. COLLECTE

Sensor → événements bruts → buffer local

2. ANALYSE LOCALE

Buffer local → moteur local → score de risque local
Si score > seuil local → réponse locale immédiate

3. TRANSMISSION

Buffer local → gRPC chiffré → Thymus Core
(avec compression, par lots, toutes les 5-30 secondes)

4. ANALYSE CENTRALE

Core reçoit les événements de tous les sensors
→ Corrélation multi-machines
→ Mise à jour du modèle du Soi
→ Scoring global
→ Path Intelligence

5. DECISION

Score global → moteur de réponse → actions coordonnées
→ Commandes envoyées aux sensors concernés

6. APPRENTISSAGE

Résultat de l'incident → mise à jour mémoire immunitaire
→ Ajustement des règles (sélection clonale)
→ Mise à jour du modèle du Soi

6.3 Stack technique

Composant	Technologie	Justification
Sensor	Rust (tokio, netlink, eBPF)	Performance, sécurité mémoire, empreinte minimale (< 20 Mo RAM)
Core	Rust (axum, tokio)	Binaire unique, pas de dépendances runtime
Communication	gRPC + TLS mutuel	Bidirectionnel, performant, authentifié
Stockage événements	DuckDB embarqué	Analytique colonnes, zero-config, un fichier
Configuration / Profils	SQLite	Embarqué, transactionnel, fiable
Frontend	Nuxt 3 + TypeScript	Servi par le Core, temps réel via WebSocket
Graphe réseau	Visualisation : D3.js	Cartographie interactive
Alertes	Webhooks + SMS (Orange API)	Adapté au contexte local

7. COMPOSANTS TECHNIQUES

7.1 Thymus Sensor

Responsabilités

Le Sensor est l'agent déployé sur chaque machine. C'est la cellule de base du système immunitaire.

Fonctions du Sensor :

1. OBSERVER
 - Capturer les connexions réseau (via netlink / eBPF sur Linux, ETW sur Windows)
 - Lister les processus actifs et leurs connexions
 - Surveiller les événements système (fichiers, registre, services)
 - Mesurer les volumes de données
2. CONNAITRE SON ADN
 - Stocker une copie compacte du profil de sa machine
 - Recevoir les mises à jour de profil depuis le Core
 - Comparer en temps réel le comportement actuel au profil
3. REAGIR LOCALEMENT
 - Appliquer les réponses de niveau 1-4 sans attendre le Core
 - Exécuter les commandes reçues du Core
 - Réverser les actions quand le Core le demande
4. COMMUNIQUER
 - Envoyer les événements au Core (par lots, compressés)
 - Recevoir les mises à jour de profil et de règles
 - Signaler les anomalies détectées localement
 - Continuer à fonctionner si le Core est injoignable

Contraintes de performance

Empreinte mémoire : < 20 Mo en régime normal, < 50 Mo en pic
Utilisation CPU : < 2% en régime normal, < 5% en pic
Taille du binaire : < 10 Mo
Latence de détection : < 100 ms pour la détection locale
Buffer local : 24h d'événements si Core injoignable

Collecte réseau (Linux)

```
// Utilisation de netlink pour capturer les connexions
// sans intercepter le trafic (pas de packet capture)

struct NetworkEvent {
    timestamp: u64,
    source_ip: IpAddr,
    source_port: u16,
    dest_ip: IpAddr,
    dest_port: u16,
    protocol: Protocol,
    bytes_sent: u64,
    bytes_recv: u64,
    process_pid: u32,
    process_name: String,
    process_user: String,
    connection_state: ConnState,
    duration_ms: u64,
}
```

Collecte processus

```
struct ProcessEvent {
    timestamp: u64,
    pid: u32,
    ppid: u32,
    name: String,
    exe_path: String,
    cmdline: String,
    user: String,
    cpu_percent: f32,
    memory_bytes: u64,
    open_files: u32,
    network_connections: u32,
    event_type: ProcessEventType, // Started, Stopped, Modified
}
```

Collecte système

```
struct SystemEvent {
    timestamp: u64,
    event_type: SystemEventType,
    source: String,
    details: String,
    severity: Severity,
}

enum SystemEventType {
    FileModified { path: String, hash_before: String, hash_after: String },
    ServiceChanged { name: String, old_state: String, new_state: String },
    UserCreated { username: String },
    PrivilegeEscalation { user: String, method: String },
    CronModified { entry: String },
    KernelModuleLoaded { name: String },
}
```

7.2 Thymus Core

Responsabilités

Le Core est le cerveau du système immunitaire. Il reçoit les données de tous les Sensors, corrèle, analyse, et coordonne les réponses.

Fonctions du Core :

1. RECEVOIR

- Ingérer les événements de tous les Sensors
- Valider l'authenticité (TLS mutuel)
- Stocker dans DuckDB

2. MODELE DU SOI

- Construire et maintenir le modèle d'identité du réseau
- Calculer les profils ADN de chaque machine
- Mettre à jour les profils en continu (évolution lente)
- Distribuer les profils compacts aux Sensors

3. DETECTION GLOBALE

- Corréler les événements entre machines
- Détecter les mutations multi-machines (déplacement latéral)
- Calculer les scores de risque globaux
- Exécuter le Path Intelligence

4. MEMOIRE IMMUNITAIRE

- Stocker et consulter les Cellules Mémoire
- Gérer la Tolérance (faux positifs)
- Exécuter la Sélection Clonale (optimisation des règles)

5. REPONSE COORDONNEE

- Décider des actions à prendre
- Envoyer des commandes aux Sensors
- Coordonner les isolations multi-machines
- Appliquer la réponse graduée

6. INTERFACE

- Servir le dashboard (Nuxt 3)
- API REST pour l'intégration
- WebSocket pour le temps réel
- Génération des rapports

Architecture interne du Core



- API REST
 - WebSocket (temps réel)
 - Sert le frontend Nuxt 3
-

8. MODELES DE DONNEES

8.1 Identité d'une machine (ADN complet)

```

struct MachineIdentity {
    machine_id: String,
    hostname: String,
    first_seen: u64,

    // ADN technique
    technical: TechnicalDNA {
        os: String,
        os_version: String,
        cpu_cores: u8,
        ram_mb: u32,
        services: Vec<ServiceInfo>,
        listening_ports: Vec<u16>,
        installed_software: Vec<SoftwareInfo>,
    },

    // ADN relationnel
    relational: RelationalDNA {
        known_peers: Vec<PeerProfile>,
        // Pour chaque pair : IP, ports, protocole, direction, volume moyen,
        // fréquence, première et dernière communication

        organ: String, // groupe fonctionnel (RH, Finance, IT, etc.)
        role: MachineRole, // Server, Workstation, Infrastructure
    },

    // ADN temporel
    temporal: TemporalDNA {
        active_hours: (u8, u8), // ex: (8, 18) = 08h-18h
        active_days: Vec<Weekday>, // ex: [Lun, Mar, Mer, Jeu, Ven]
        avg_hourly_volume: [u64; 24], // volume moyen par heure
        avg_daily_connections: f64,
        avg_daily_volume_bytes: u64,
        seasonal_patterns: Vec<SeasonalPattern>,
    },

    // Métadonnées du profil
    profile_maturity: f64, // 0.0 -> 1.0
    last_updated: u64,
    observation_days: u32,
}

struct PeerProfile {
    peer_ip: IpAddr,
    peer_hostname: Option<String>,
    ports: Vec<u16>,
    protocols: Vec<Protocol>,
    direction: ConnectionDirection, // Outgoing, Incoming, Both
    avg_daily_volume: u64,
    avg_daily_connections: f64,
    first_seen: u64,
    last_seen: u64,
    confidence: f64,
}

```

```
struct SeasonalPattern {
    description: String,          // ex: "Clôture mensuelle"
    frequency: PatternFrequency,  // Weekly, Monthly, Quarterly, Annual
    expected_dates: Vec<DateRange>,
    volume_multiplier: f64,       // ex: 3.0 = volume x3 pendant cette période
    connection_multiplier: f64,
}
```

8.2 Modèle du Soi

```
struct SelfModel {
    organization_id: String,
    created_at: u64,
    last_updated: u64,

    // Toutes les cellules
    machines: Vec<MachineIdentity>,

    // Le graphe des relations normales
    relation_graph: RelationGraph {
        edges: Vec<RelationEdge>,
        // Chaque edge = une relation habituelle entre deux machines
        // avec son volume, fréquence, protocoles, horaires
    },

    // Les organes (groupes fonctionnels)
    organs: Vec<Organ> {
        name: String,          // "RH", "Finance", "IT", "Production"
        machines: Vec<String>, // machine_ids
        internal_density: f64, // densité des communications internes
        external_peers: Vec<String>, // organes avec lesquels il communique
    },

    // Métabolisme global
    metabolism: NetworkMetabolism {
        total_daily_volume: u64,
        total_daily_connections: u64,
        internet_egress_daily: u64,
        peak_hours: (u8, u8),
        quiet_hours: (u8, u8),
    },

    // Confiance du modèle
    confidence: f64,
    observation_days: u32,

    // Tolérance
    tolerance_entries: Vec<ToleranceEntry>,
}
```

8.3 Mutation

```
struct Mutation {
    id: uuid,
    detected_at: u64,

    // Quelle machine est en mutation
    machine_id: String,

    // Dimensions affectées
    dimensions: Vec<MutationDimension>,

    // Score de risque
    risk_score: f64, // 0.0 -> 1.0

    // Contribution de chaque couche
    innate_score: f64,
    adaptive_score: f64,

    // Détails
    details: Vec<MutationDetail>,

    // Si correspondance mémoire
    memory_match: Option<MemoryCellId>,

    // Réponse appliquée
    response: Option<ResponseAction>,

    // Résolution
    status: MutationStatus, // Active, Investigating, Resolved, FalsePositive
    resolved_at: Option<u64>,
    resolution_notes: Option<String>,
}

enum MutationDimension {
    Technical,    // nouveau service, nouveau port, nouveau logiciel
    Relational,   // nouvelle connexion, nouveau pair, nouvelle direction
    Temporal,     // horaire inhabituel, fréquence inhabituelle
    Volumetric,   // volume anormal
}

struct MutationDetail {
    dimension: MutationDimension,
    description_fr: String,
    expected_value: String,
    observed_value: String,
    deviation_sigma: f64, // nombre d'écarts-types
}
```


8.4 Cellule mémoire

```
struct MemoryCell {
    id: uuid,
    created_at: u64,

    // Signature de la mutation
    mutation_signature: MutationSignature {
        dimensions: Vec<MutationDimension>,
        deviation_profile: Vec<(MutationDimension, f64, f64)>,
        // (dimension, déviation min, déviation max)
        progression: Vec<ProgressionStep>,
        // séquence temporelle de la mutation
        duration_range: (Duration, Duration),
    },

    // Contexte de l'incident original
    original_context: IncidentContext {
        attack_type: Option<String>,    // si identifié
        origin: String,                 // machine source
        targets: Vec<String>,           // machines ciblées
        path: Vec<String>,              // chemin emprunté
        data_at_risk: Option<String>,   // estimation des données en jeu
    },

    // Réponse qui a fonctionné
    effective_response: Vec<ResponseAction>,
    time_to_containment: Duration,

    // Efficacité
    times_matched: u32,
    true_matches: u32,
    false_matches: u32,
    effectiveness: f64,

    // Source
    source: MemorySource, // Local, Collective, Vaccination
}
```

9. ALGORITHMES FONDAMENTAUX

9.1 Scoring d'anomalie

Le scoring combine la couche innée et la couche adaptative.

Score inné

Pour chaque événement :

1. Vérifier les règles innées (liste prédéfinie)
2. Si match → score_inné = sévérité de la règle (0.0 -> 1.0)
3. Si pas de match → score_inné = 0.0

Score adaptatif

```

Pour chaque événement impliquant la machine M :

score_relationnel = 0.0
Si destination jamais vue dans le profil de M :
    score_relationnel = 0.8
Si destination vue mais port/protocole inhabituel :
    score_relationnel = 0.4

score_volumetrique = 0.0
Si volume_actuel > (moyenne + 3 * écart_type) :
    score_volumetrique = min(1.0, (volume_actuel - moyenne) / (10 * écart_type))

score_temporel = 0.0
Si heure_actuelle hors plage_habituelle :
    score_temporel = 0.6
Si jour_actuel hors jours_habituels :
    score_temporel += 0.3

score_technique = 0.0
Si nouveau service détecté :
    score_technique = 0.5
Si nouveau port en écoute :
    score_technique = 0.6

score_adaptatif = max(
    score_relationnel * 0.35 +
    score_volumetrique * 0.30 +
    score_temporel * 0.20 +
    score_technique * 0.15,

    max(score_relationnel, score_volumetrique)
    // Le max individuel évite qu'une mutation massive dans une
    // dimension soit diluée par les autres
)

```

Score combiné

```

score_final = max(score_inné, score_adaptatif)

// Si les deux couches signalent quelque chose, le score augmente
Si score_inné > 0.3 ET score_adaptatif > 0.3 :
    score_final = min(1.0, score_final * 1.3)

```

Consultation mémoire

```
Si score_final > 0.4 :
    Chercher dans la mémoire immunitaire une signature similaire

    Si correspondance trouvée (similarité > 0.7) :
        Appliquer directement la réponse mémorisée
        Temps de réponse : secondes au lieu de minutes
```

Vérification tolérance

```
Avant de déclencher une alerte :

1. Vérifier les ToleranceEntry pour cette machine
2. Vérifier les contextes actifs (maintenance, migration, etc.)
3. Si le comportement est toléré :
    → Réduire le score_final de 50%
    → Ou annuler l'alerte si la tolérance est totale
```

9.2 Détection de déplacement latéral

Le déplacement latéral est la technique la plus dangereuse : l'attaquant se déplace de machine en machine pour atteindre sa cible.

```
Algorithme :

1. Détecter une première mutation sur Machine A
   (ex: connexion inhabituelle vers Machine B)

2. Dans les N minutes suivantes, surveiller Machine B :
   - Est-ce que B commence aussi à muter ?
   - Est-ce que B contacte des machines qu'elle ne contacte jamais ?

3. Si B mute à son tour et contacte Machine C :
   → Construire le chemin : A → B → C
   → Calculer un score de chaîne :
       score_chaine = 1.0 - ∏(1.0 - score_mutation_i)
       // Le score augmente avec chaque maillon

4. Si score_chaine > seuil :
   → Alerte : déplacement latéral détecté
   → Visualiser le chemin sur la cartographie
   → Réponse coordonnée sur toutes les machines du chemin

Fenêtre de corrélation : configurable (défaut : 30 minutes)
```

9.3 Détection d'exfiltration

Algorithme :

1. Surveiller le volume sortant de chaque machine vers Internet
2. Calculer le ratio :
$$\text{ratio} = \text{volume_sortant_actuel} / \text{volume_sortant_habituel}$$
3. Appliquer un scoring progressif :
Si ratio > 3 : score = 0.4 (inhabituel)
Si ratio > 10 : score = 0.7 (très inhabituel)
Si ratio > 50 : score = 0.9 (exfiltration probable)
4. Facteurs aggravants :
+ 0.2 si la destination est nouvelle
+ 0.2 si l'heure est inhabituelle
+ 0.1 si le protocole est inhabituel
+ 0.3 si précédé d'un déplacement latéral
5. Si score > 0.7 :
→ Réponse immédiate : throttle du débit sortant
→ Alerte critique
→ Path Intelligence : d'où viennent ces données ?

9.4 Sélection clonale (auto-optimisation)

Exécuté chaque semaine :

Pour chaque règle de détection R :

```
precision = true_positives / (true_positives + false_positives)
recall = true_positives / (true_positives + false_negatives)
effectiveness = 2 * (precision * recall) / (precision + recall)
// F1-score
```

Si effectiveness > 0.8 :

```
R.threshold *= 0.9    // plus sensible
R.status = Amplified
Propager R aux sensors qui ne l'ont pas encore
```

Si effectiveness entre 0.4 et 0.8 :

```
// Pas de changement
R.status = Active
```

Si effectiveness entre 0.2 et 0.4 :

```
R.threshold *= 1.2    // moins sensible
R.status = Attenuated
Augmenter le niveau d'alerte minimum
```

Si effectiveness < 0.2 :

```
R.status = Eliminated
Archiver dans l'historique
Retirer de l'évaluation active
```

Résultat : les bonnes règles deviennent plus sensibles,
les mauvaises disparaissent naturellement.

10. CARTOGRAPHIE VIVANTE

10.1 Le graphe du réseau

Thymus construit et maintient en permanence un graphe du réseau. Ce n'est pas un scan ponctuel -- c'est un organisme vivant.

Graphe :

Noeuds = machines

Arêtes = relations (communications régulières)

Chaque noeud possède :

- Son ADN (technique, relationnel, temporel)
- Son état actuel (normal, en surveillance, en alerte, isolé)
- Son organe d'appartenance

Chaque arête possède :

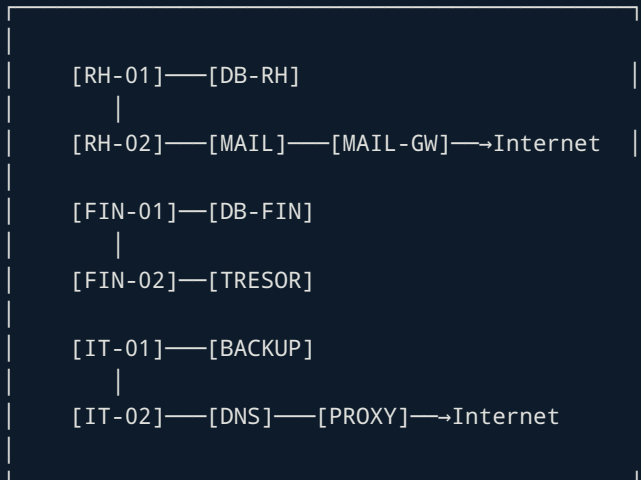
- Le volume habituel
- La fréquence
- Les protocoles
- L'état (normal, anormal, nouveau)

10.2 Visualisation

Le dashboard affiche le réseau comme un organisme :

Vue normale :

=====



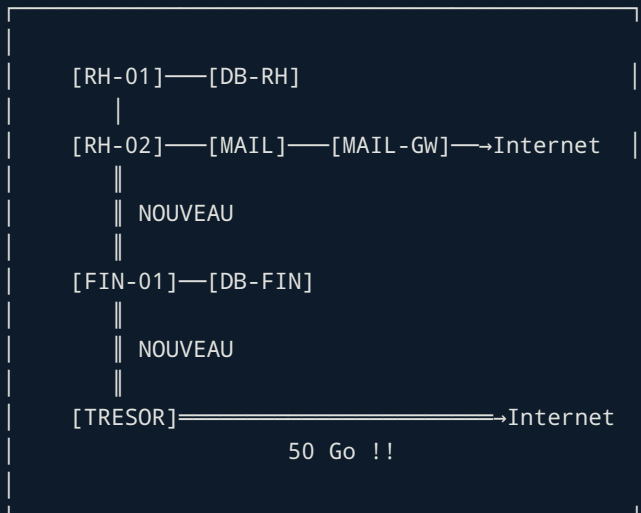
● Vert = normal

Toutes les connexions sont habituelles.

Le réseau est sain.

Vue en alerte :

=====



● Rouge = mutation détectée

|| = connexion nouvelle (jamais vue)

Chemin de l'attaque :

RH-02 → FIN-01 → TRESOR → Internet

Interprétation :

Compromission de RH-02

Déplacement latéral vers Finance

Exfiltration depuis TRESOR

10.3 Organes

Le graphe identifie automatiquement les groupes fonctionnels (organes) par analyse de la densité des communications :

```
Organe "Ressources Humaines" :  
  Machines : RH-01, RH-02, DB-RH  
  Communications internes : 85% du trafic  
  Communications externes : MAIL (15%)  
  
Organe "Finance" :  
  Machines : FIN-01, FIN-02, DB-FIN, TRESOR  
  Communications internes : 90% du trafic  
  Communications externes : ERP (10%)  
  
Organe "Infrastructure" :  
  Machines : DNS, PROXY, BACKUP, MAIL-GW  
  Communications : avec tous les organes
```

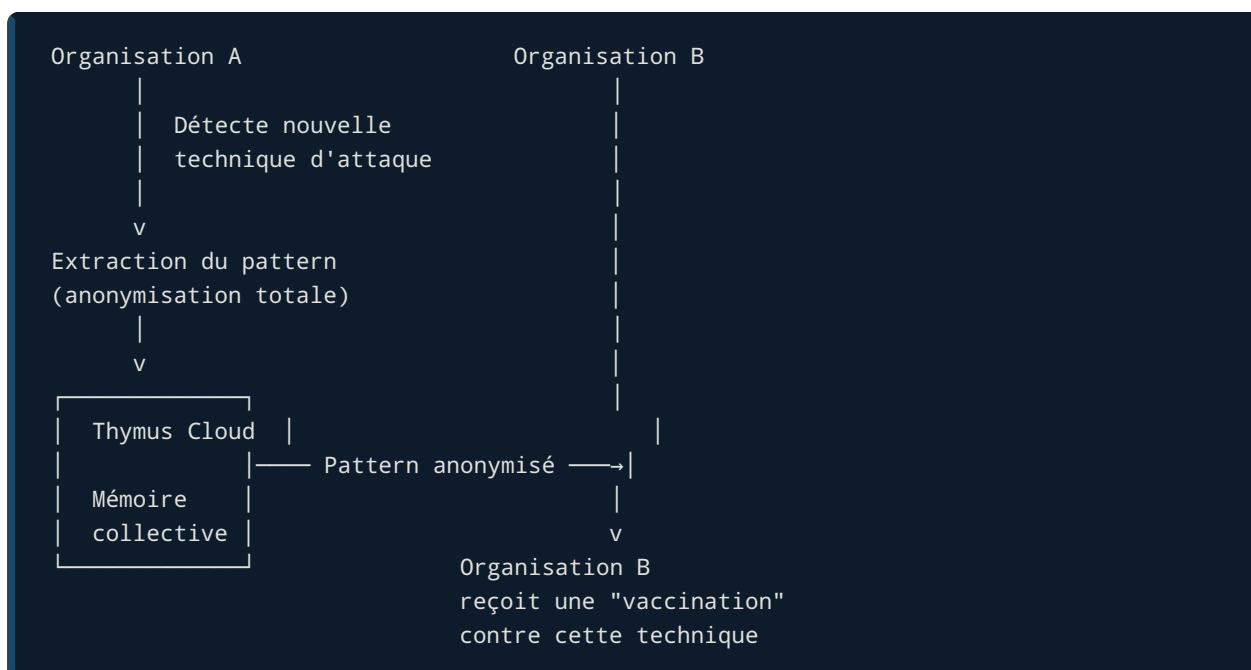
Quand une communication traverse les frontières d'un organe de manière inhabituelle (RH-02 → FIN-01), c'est un signal fort.

11. SYSTEME IMMUNITAIRE COLLECTIF

11.1 Vision

A long terme, Thymus permet à plusieurs organisations de bénéficier d'une immunité collective, comme un groupe d'individus vaccinés protège l'ensemble de la population.

11.2 Fonctionnement



11.3 Garanties de confidentialité

Ce qui est partagé :

```

MutationSignature (anonymisé) :
{
  "dimensions": ["relational", "volumetric"],
  "deviation_range": [5.0, 50.0],
  "progression_steps": 3,
  "duration_hours": 4,
  "effective_response": ["throttle", "isolate"]
}
  
```

Ce qui n'est JAMAIS partagé :

- Noms de machines
- Adresses IP
- Noms d'utilisateurs
- Données transférées
- Structure du réseau
- Noms de l'organisation

Le pattern décrit la forme de la mutation, pas son contenu.

11.4 Architecture du Thymus Cloud



12. DEMONSTRATIONS

12.1 Démonstration 1 : Exfiltration de données

Scénario

Un employé du département RH reçoit un email de phishing. Il clique sur un lien, ce qui permet à un attaquant d'obtenir un accès distant à son poste.

Déroulement sans Thymus

```
Jour 1 : L'attaquant accède au poste RH-02
Jour 2 : Il explore le réseau, découvre le serveur Finance
Jour 3 : Il accède à FIN-01 avec des identifiants volés
Jour 4 : Il découvre le serveur TRESOR
Jour 5 : Il copie 50 Go de données financières
Jour 6 : Il exfiltre les données via HTTPS
Jour 7 : L'organisation découvre la fuite (ou pas)

Bilan : données financières sensibles exfiltrées.
Détection : aucune (identifiants valides, HTTPS, pas de malware).
```

Déroulement avec Thymus

Jour 1, 14h32 : L'attaquant accède au poste RH-02

Thymus observe : rien d'anormal, l'utilisateur est connecté normalement

Jour 1, 14h45 : L'attaquant scanne le réseau depuis RH-02

Thymus détecte :

- Mutation relationnelle : RH-02 tente de contacter 12 machines en 3 minutes (habituellement : 3 machines/jour)
- Score de risque : 0.65
- Action : Surveillance renforcée (niveau 1)
- Notification : admin notifié par SMS

Jour 1, 15h10 : L'attaquant tente d'accéder à FIN-01 depuis RH-02

Thymus détecte :

- Mutation relationnelle : RH-02 n'a JAMAIS communiqué avec FIN-01
- Cette connexion traverse une frontière d'organe (RH → Finance)
- Score de risque : 0.82
- Action : Limitation du débit (niveau 2) + Alerte
- Path Intelligence active : surveille la chaîne

Jour 1, 15h15 : L'attaquant tente d'accéder à TRESOR depuis FIN-01

Thymus détecte :

- Chaîne de mutations : RH-02 → FIN-01 → TRESOR
- Ce chemin n'a jamais existé dans l'historique du réseau
- Score de chaîne : 0.94
- Action : ISOLATION de RH-02 et blocage de FIN-01→TRESOR
- Alerte critique

ALERTE CRITIQUE

Déplacement latéral détecté

Chemin : RH-02 → FIN-01 → TRESOR

Ce chemin n'a jamais existé.

Probabilité de compromission : 94%

Actions prises :

- RH-02 isolé du réseau
- Connexion FIN-01 → TRESOR bloquée
- Admin notifié par SMS

Recommandation :

Vérifier le poste RH-02 avec l'utilisateur immédiatement

Bilan : attaque stoppée en 43 minutes.

Aucune donnée exfiltrée.

L'attaquant n'a jamais atteint TRESOR.

Apprentissage post-incident

Thymus crée une Cellule Mémoire :

```
{  
  mutation_signature: {  
    dimensions: [Relational, Relational, Volumetric],  
    progression: [scan → cross-organ → high-value-target],  
    duration: 43 minutes  
  },  
  effective_response: [Monitor, Throttle, Isolate],  
  time_to_containment: 43 minutes  
}
```

Si un scénario similaire se reproduit (même pattern de progression),
Thymus répondra en secondes au lieu de minutes.

12.2 Démonstration 2 : Menace interne

Scénario

Un employé de la comptabilité, mécontent, décide de copier des documents sensibles avant de quitter l'entreprise.

Déroulement avec Thymus

Semaine 1 :

L'employé commence à accéder à des dossiers qu'il ne consulte jamais.

Thymus détecte :

- Mutation relationnelle : COMPTA-03 accède à des répertoires du serveur de fichiers qu'il n'a jamais consultés
- Score : 0.35 (faible, pourrait être légitime)
- Action : surveillance silencieuse

Semaine 2 :

L'employé commence à copier des fichiers sur une clé USB.

Thymus détecte :

- Mutation volumétrique : volume local de COMPTA-03 augmente (copies vers périphérique USB)
- Combiné avec la mutation relationnelle de la semaine précédente
- Score cumulé : 0.55
- Action : surveillance renforcée + entrée dans le rapport hebdo

Semaine 3 :

L'employé tente d'envoyer 15 Go par email via le webmail personnel.

Thymus détecte :

- Mutation volumétrique majeure : volume sortant x50
- Destination : webmail externe (pas le mail professionnel)
- Horaire : 22h (hors horaires habituels pour COMPTA-03)
- Score : 0.88
- Action : throttle immédiat + alerte admin

ALERTE - EXFILTRATION PROBABLE

Machine : COMPTA-03

Utilisateur : [nom]

Historique des mutations :

- Semaine 1 : accès inhabituels aux fichiers
- Semaine 2 : copies massives sur USB
- Semaine 3 : tentative d'envoi de 15 Go
vers webmail externe
à 22h (hors horaires)

Score de risque cumulé : 0.88

Action prise :

Débit sortant limité à 1 Mo/s

Recommandation :

Incident potentiel de fuite de données.

Impliquer le responsable sécurité et les RH.

Point clé

Aucun malware. Aucune signature. Aucun IOC. L'utilisateur utilise ses propres identifiants et des outils légitimes (USB, webmail). Un EDR classique ne voit rien. Thymus voit la mutation progressive du comportement.

12.3 Démonstration 3 : Ransomware

Scénario

Un ransomware pénètre le réseau et tente de chiffrer les fichiers sur plusieurs serveurs.

Déroulement avec Thymus

Minute 0 : Le ransomware s'exécute sur IT-02

Thymus détecte (couche innée) :

- Processus inconnu lancé depuis /tmp
- Score inné : 0.6

Thymus détecte (couche adaptative) :

- IT-02 commence à accéder massivement à des fichiers sur le serveur de fichiers (1000 fichiers/minute vs 50 habituels)
- Score adaptatif : 0.75

Score combiné : 0.85

Action : blocage des connexions vers le serveur de fichiers

Minute 1 : Le ransomware tente de se propager

Thymus détecte :

- IT-02 tente de se connecter à toutes les machines du réseau sur le port 445 (SMB)
- Mutation relationnelle explosive : 30 nouvelles connexions en 60s
- Score : 0.95

Action : ISOLATION COMPLETE de IT-02

Minute 2 : Alerte

ALERTE CRITIQUE - RANSOMWARE PROBABLE

Machine : IT-02

Indicateurs :

- Processus inconnu depuis /tmp
- Accès massif aux fichiers (1000/min)
- Tentative de propagation SMB (30 machines)

Actions prises :

- IT-02 isolé du réseau (toutes connexions)
- Serveur de fichiers : accès en lecture seule
- Toutes les machines : surveillance maximale

Aucune autre machine n'a été touchée.

Temps de confinement : 2 minutes.

Machines touchées : 1 sur 50.

Données chiffrées : aucune (bloqué avant le chiffrement massif).

12.4 Démonstration 4 : Système immunitaire collectif

Scénario

Une nouvelle technique d'attaque est utilisée contre le Ministère A. Grâce au système collectif, la Banque B est protégée avant même d'être attaquée.

Jour 1 : Ministère A

Nouvelle technique détectée :

L'attaquant utilise des requêtes DNS pour exfiltrer des données (DNS tunneling avec des sous-domaines encodés en base64)

Thymus Ministère A détecte la mutation :

- Volume DNS x100
- Sous-domaines de longueur inhabituelle
- Pattern : requêtes DNS régulières toutes les 5 secondes

Incident résolu.

Thymus crée une Cellule Mémoire et envoie le pattern anonymisé au Thymus Cloud :

```
{
  "type": "exfiltration",
  "channel": "dns",
  "indicators": {
    "dns_volume_multiplier": 100,
    "subdomain_length_avg": 45,
    "query_interval_seconds": 5,
    "encoding": "base64_like"
  }
}
```

Jour 2 : Thymus Cloud

Le pattern est validé, déduplicué, et distribué à tous les déploiements Thymus.

Chaque déploiement reçoit une "vaccination" : une nouvelle règle de détection pour ce type de DNS tunneling.

Jour 3 : Banque B

Un attaquant tente la même technique contre la Banque B.

Thymus Banque B possède déjà la vaccination.

Détection en 30 secondes (au lieu de plusieurs heures). L'attaque est bloquée avant que la moindre donnée ne sorte.

La Banque B n'a jamais vu cette attaque. Mais elle était déjà immunisée.

13. INTERFACE ET EXPERIENCE UTILISATEUR

13.1 Philosophie de l'interface

L'interface Thymus n'est pas un dashboard SIEM classique rempli de graphiques illisibles.

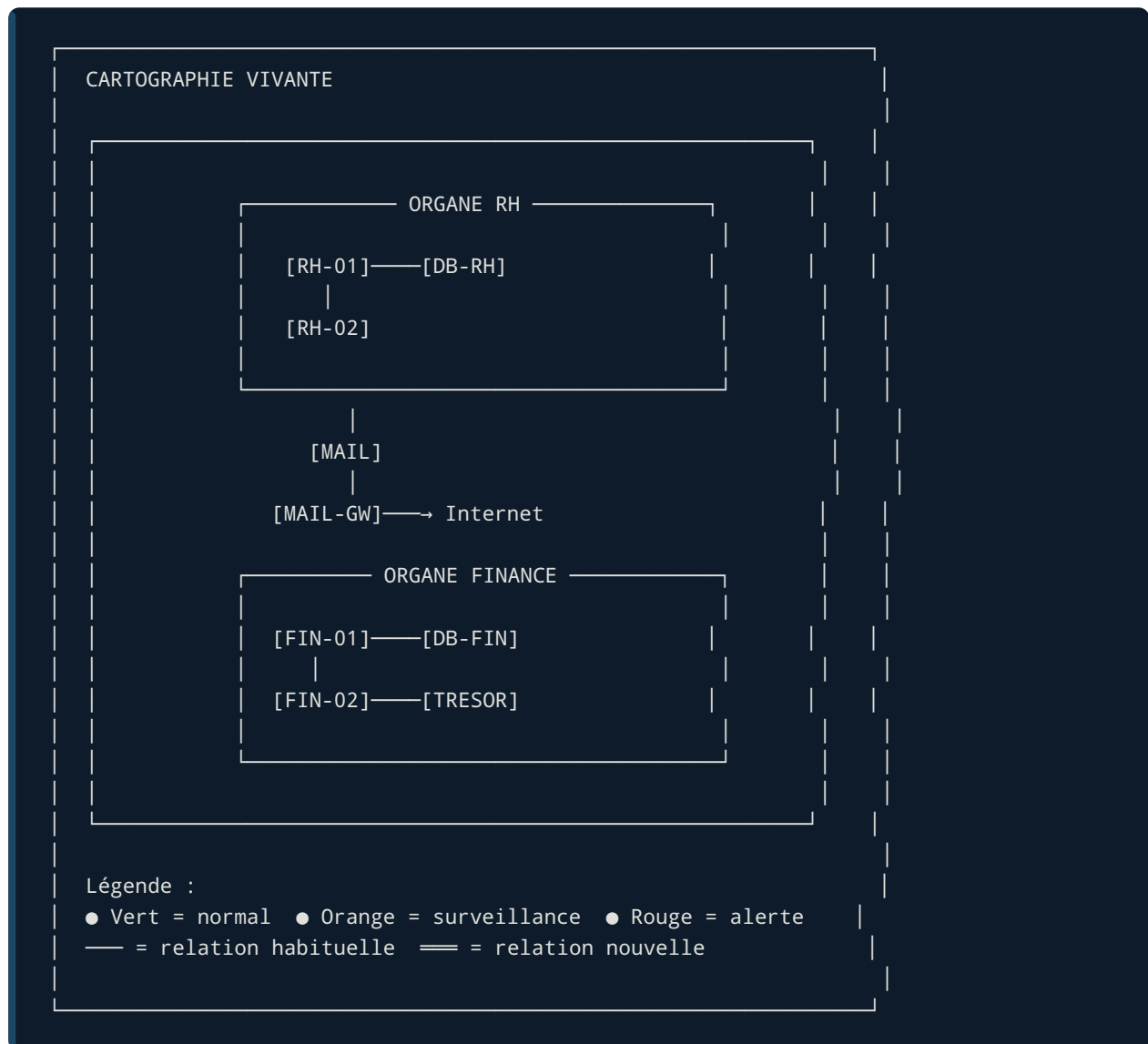
L'interface est conçue pour deux publics :

1. **Le directeur / responsable non-technique** : a besoin de savoir si tout va bien, en une phrase
2. **L'administrateur réseau / analyste** : a besoin de comprendre ce qui se passe et d'agir

13.2 Vue principale : l'état de l'organisme



13.3 Vue cartographie



13.4 Vue mutation active

MUTATION ACTIVE #2024-0847

Sévérité :  82%

Statut : En investigation

Machine : RH-02 (Ressources Humaines)

Détectée : 2 juin 2026 à 15h10

MUTATIONS DETECTEES :

1. Relationnelle

RH-02 a contacté FIN-01 pour la première fois.
Cette connexion traverse la frontière RH → Finance.
Déviation : relation inédite (score: 0.8)

2. Temporelle

Activité détectée à 15h10 sur un port inhabituel.
Déviation : +3.2 sigma (score: 0.4)

CHEMIN DETECTE :

[RH-02] → [FIN-01] → [TRESOR] (bloqué)
15h10 15h12 15h15

REPONSE APPLIQUEE :

15h10 : Surveillance renforcée sur RH-02
15h12 : Limitation du débit RH-02 → FIN-01
15h15 : Isolation de RH-02 + blocage FIN-01 → TRESOR

MEMOIRE IMMUNITAIRE :

Correspondance partielle avec Cellule #MC-0023
(incident similaire il y a 4 mois, même type de progression)

[Marquer résolu] [Faux positif] [Escalader]

13.5 Rapport automatique

RAPPORT HEBDOMADAIRE Thymus
Ministère des Finances - Semaine 22/2026

SANTE DE L'ORGANISME : BONNE

Résumé :

- 32 machines surveillées (aucun changement)
- 1 mutation détectée et contenue mardi
- 0 donnée exfiltrée
- Modèle de confiance : 87%

Incident de la semaine :

Mardi 2 juin à 15h10, le poste RH-02 a présenté un comportement inhabituel : tentative d'accès aux serveurs Finance sans précédent. Thymus a isolé la machine en 43 minutes. Investigation en cours.

Immunité :

- 3 nouvelles règles adaptatives créées cette semaine
- 1 règle atténuée (trop de faux positifs)
- 2 vaccinations reçues du réseau collectif

Recommandations :

- Vérifier le poste RH-02 avec l'utilisateur
- Confirmer que la migration ERP prévue vendredi est déclarée dans Thymus (pour éviter les faux positifs)

14. ROADMAP TECHNIQUE

Phase 1 : Fondations (Mois 1-3)

Objectif

Construire le Sensor et le Core minimal. Pouvoir observer un réseau de 5 machines et construire des profils.

Livrables

```
Mois 1 : Sensor v0.1
├─ Collecte réseau via netlink (Linux uniquement)
├─ Collecte processus via /proc
├─ Buffer local (si Core injoignable)
├─ Communication gRPC avec le Core
└─ Tests sur 2-3 VMs

Mois 2 : Core v0.1
├─ Réception des événements (gRPC server)
├─ Stockage dans DuckDB
├─ Construction des profils ADN (technique + relationnel + temporel)
├─ API REST basique
└─ Phase Thymus fonctionnelle

Mois 3 : Détection v0.1
├─ Scoring d'anomalie basique (z-scores)
├─ Couche innée (10 règles fondamentales)
├─ Couche adaptative (comparaison au profil)
├─ Alertes par webhook
└─ Demo sur lab de 5 VMs
```

Critère de succès

Sur un lab de 5 machines virtuelles, simuler une exfiltration et vérifier que Thymus :

1. Construit correctement les profils après 7 jours
2. Détecte la mutation relationnelle
3. Détecte la mutation volumétrique
4. Génère une alerte avec un score > 0.7

Phase 2 : Intelligence (Mois 4-6)

Objectif

Ajouter les mécanismes immunitaires avancés et le dashboard.

Livrables

Mois 4 : Réponse locale

- └─ Sensor : détection locale (copie compacte de l'ADN)
- └─ Sensor : réponse locale (niveaux 1-4)
- └─ Sensor : fonctionnement offline
- └─ Core : détection de déplacement latéral
- └─ Core : Path Intelligence basique

Mois 5 : Mémoire et tolérance

- └─ Mémoire immunitaire (création de Cellules Mémoire)
- └─ Consultation de la mémoire lors des détections
- └─ Tolérance : gestion des faux positifs
- └─ Tolérance : contextes (maintenance, migration)
- └─ Sélection clonale (auto-optimisation hebdomadaire)

Mois 6 : Dashboard

- └─ Interface Nuxt 3 : état de l'organisme
- └─ Cartographie réseau interactive (D3.js)
- └─ Vue des mutations actives
- └─ Rapport automatique quotidien et hebdomadaire
- └─ Alertes SMS (Orange API)
- └─ Demo complète

Critère de succès

Démo complète montrant :

1. Phase Thymus → construction de l'identité
 2. Détection d'exfiltration avec chemin d'attaque
 3. Réponse automatique graduée
 4. Mémoire : deuxième attaque similaire détectée plus vite
 5. Dashboard affichant tout en temps réel
-

Phase 3 : Production (Mois 7-9)

Objectif

Rendre Thymus déployable en production sur un vrai réseau.

Livrables

Mois 7 : Durcissement

- Agent Windows (ETW pour la collecte)
- TLS mutuel pour toutes les communications
- Tests de charge : 100 machines, 10M événements/jour
- Optimisation DuckDB : rétention, compression, rotation
- Documentation d'installation

Mois 8 : Opérationnel

- Installation en une commande (script d'installation)
- Mise à jour automatique des Sensors
- Backup et restauration du Core
- Monitoring de la santé de Thymus lui-même
- Guide opérationnel pour les administrateurs

Mois 9 : Pilote

- Déploiement chez le premier partenaire pilote
- Phase Thymus en conditions réelles
- Ajustements basés sur les retours terrain
- Correction des faux positifs
- Rapport de pilote

Critère de succès

Thymus déployé et fonctionnel chez un partenaire pilote pendant 30 jours, avec :

1. Moins de 5 faux positifs par semaine après 2 semaines
2. Au moins 1 anomalie réelle détectée (ou confirmée qu'il n'y en a pas)
3. Rapport hebdomadaire automatique jugé utile par l'administrateur
4. Aucune interruption de service causée par Thymus

Phase 4 : Avancé (Mois 10-12)

Objectif

Ajouter le système collectif et préparer le déploiement multi-organisations.

Livrables

Mois 10 : Système collectif

- └─ Thymus Cloud : réception de patterns anonymisés
- └─ Thymus Cloud : distribution de vaccinations
- └─ Protocole d'anonymisation vérifié
- └─ 2e et 3e partenaires pilotes

Mois 11 : Intelligence avancée

- └─ Détection de chaînes d'attaque complexes (> 3 étapes)
- └─ Prédiction de la progression probable d'une attaque
- └─ Recommandations automatiques de segmentation réseau
- └─ Identification automatique des organes

Mois 12 : Consolidation

- └─ Retours de tous les pilotes intégrés
- └─ Performance validée sur 3 réseaux réels
- └─ Documentation complète
- └─ Version 1.0 stable
- └─ Préparation du lancement commercial

Phase 5 : Expansion (Année 2)

Trimestre 1 :

- └─ Lancement commercial
- └─ Offre Starter (gratuit, < 10 machines)
- └─ Offre Pro (payant, 10-100 machines)
- └─ Offre Souverain (sur devis, 100+ machines)

Trimestre 2 :

- └─ Intégration IA (résumé des incidents en langage naturel)
- └─ Prédiction de risques basée sur la mémoire collective
- └─ Expansion premiers clients payants

Trimestre 3 :

- └─ Thymus Cloud national (si partenariat gouvernemental)
- └─ Tableau de bord multi-organisations
- └─ API d'intégration avec les outils existants (SIEM, ticketing)

Trimestre 4 :

- └─ Expansion UEMOA
 - └─ Agent macOS
 - └─ Conteneurs et cloud (AWS, Azure, GCP)
 - └─ Certifications éventuelles
-

15. EQUIPE ET ORGANISATION

15.1 Equipe minimale (Mois 1-6)

Rôle	Compétences requises	Allocation
Lead Système / Rust	Rust senior, réseaux bas niveau, eBPF, netlink, sécurité	Temps plein
Développeur Backend Rust	Rust intermédiaire+, axum, tokio, gRPC, DuckDB	Temps plein
Développeur Frontend	Nuxt 3, TypeScript, D3.js, WebSocket	Mi-temps → temps plein
Consultant Sécurité	Pentest, red team, connaissance des techniques d'attaque	Ponctuel (1-2j/mois)

15.2 Equipe étendue (Mois 7-12)

Ajout de :

Rôle	Compétences requises	Allocation
Développeur Système Windows	Rust + Windows, ETW, API Windows	Temps plein
DevOps / Infra	Déploiement, monitoring, scripting	Mi-temps

15.3 Compétences critiques

Le succès du projet repose sur la maîtrise de :

1. **Rust système bas niveau** : netlink, eBPF, interaction noyau -- c'est le cœur technique
2. **Sécurité offensive** : comprendre les techniques d'attaque pour savoir quoi détecter
3. **Statistiques appliquées** : z-scores, distributions, détection d'anomalies sans IA
4. **Théorie des graphes** : pour la cartographie et le Path Intelligence

16. POSITIONNEMENT STRATEGIQUE

16.1 Ce que Thymus n'est PAS

- Thymus n'est pas un antivirus
- Thymus n'est pas un firewall
- Thymus n'est pas un SIEM
- Thymus n'est pas un EDR classique
- Thymus n'est pas un produit d'IA

16.2 Ce que Thymus EST

Thymus est le premier système immunitaire numérique complet :

- Il connaît l'identité du réseau (Soi vs Non-Soi)
- Il apprend de chaque incident (Lymphocytes T mémoire)
- Il s'auto-optimise (Sélection clonale)
- Il tolère les comportements rares légitimes (Tolérance immunitaire)
- Il réagit localement avant de réagir globalement (Réponse inflammatoire)
- Il possède des défenses innées ET adaptatives (Immunité duale)
- Il permet l'immunité collective entre organisations

16.3 La phrase de positionnement

Thymus ne protège pas contre les attaques connues. Thymus détecte les mutations impossibles dans l'identité vivante du réseau.

16.4 Avantages concurrentiels

Dimension	Solutions classiques	Thymus
Approche	Cherche les attaques	Connaît l'identité du réseau
Nouveauté	Dépend des mises à jour de signatures	Détecte l'inconnu par nature
Connectivité	Nécessite le cloud	Fonctionne offline
Autonomie	Nécessite un SOC	Produit des rapports lisibles
Evolution	Statique (règles manuelles)	S'auto-optimise
Prix	15-25 USD/machine/mois	Adapté au marché africain
Souveraineté	Données chez le fournisseur US	Données locales

17. ANNEXES TECHNIQUES

17.1 Format des événements gRPC

```

syntax = "proto3";

package thymus.sensor;

service SensorService {
    rpc StreamEvents(stream EventBatch) returns (stream SensorCommand);
    rpc RegisterSensor(SensorRegistration) returns (SensorConfig);
    rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse);
}

message EventBatch {
    string sensor_id = 1;
    uint64 timestamp = 2;
    repeated NetworkEvent network_events = 3;
    repeated ProcessEvent process_events = 4;
    repeated SystemEvent system_events = 5;
}

message NetworkEvent {
    uint64 timestamp = 1;
    string source_ip = 2;
    uint32 source_port = 3;
    string dest_ip = 4;
    uint32 dest_port = 5;
    Protocol protocol = 6;
    uint64 bytes_sent = 7;
    uint64 bytes_recv = 8;
    uint32 process_pid = 9;
    string process_name = 10;
    string user = 11;
}

message ProcessEvent {
    uint64 timestamp = 1;
    uint32 pid = 2;
    uint32 ppid = 3;
    string name = 4;
    string exe_path = 5;
    string cmdline = 6;
    string user = 7;
    ProcessEventType event_type = 8;
}

message SystemEvent {
    uint64 timestamp = 1;
    SystemEventType event_type = 2;
    string source = 3;
    string details = 4;
}

message SensorCommand {
    oneof command {
        UpdateProfile update_profile = 1;
        SetResponseLevel set_response = 2;
    }
}

```

```
        UpdateRules update_rules = 3;
        RequestDiagnostics diagnostics = 4;
    }
}

enum Protocol {
    TCP = 0;
    UDP = 1;
    ICMP = 2;
}

enum ProcessEventType {
    STARTED = 0;
    STOPPED = 1;
    MODIFIED = 2;
}

enum SystemEventType {
    FILE_MODIFIED = 0;
    SERVICE_CHANGED = 1;
    USER_CREATED = 2;
    PRIVILEGE_ESCALATION = 3;
    CRON_MODIFIED = 4;
    KERNEL_MODULE_LOADED = 5;
}
```

17.2 Schéma DuckDB (événements)

```

CREATE TABLE network_events (
    event_id UUID DEFAULT gen_random_uuid(),
    sensor_id VARCHAR NOT NULL,
    timestamp BIGINT NOT NULL,
    source_ip VARCHAR NOT NULL,
    source_port INTEGER,
    dest_ip VARCHAR NOT NULL,
    dest_port INTEGER NOT NULL,
    protocol VARCHAR NOT NULL,
    bytes_sent BIGINT DEFAULT 0,
    bytes_recv BIGINT DEFAULT 0,
    process_pid INTEGER,
    process_name VARCHAR,
    process_user VARCHAR,
    ingested_at BIGINT DEFAULT epoch_ms(now())
);

CREATE TABLE process_events (
    event_id UUID DEFAULT gen_random_uuid(),
    sensor_id VARCHAR NOT NULL,
    timestamp BIGINT NOT NULL,
    pid INTEGER NOT NULL,
    ppid INTEGER,
    name VARCHAR NOT NULL,
    exe_path VARCHAR,
    cmdline VARCHAR,
    username VARCHAR,
    event_type VARCHAR NOT NULL,
    ingested_at BIGINT DEFAULT epoch_ms(now())
);

CREATE TABLE system_events (
    event_id UUID DEFAULT gen_random_uuid(),
    sensor_id VARCHAR NOT NULL,
    timestamp BIGINT NOT NULL,
    event_type VARCHAR NOT NULL,
    source VARCHAR,
    details VARCHAR,
    severity VARCHAR,
    ingested_at BIGINT DEFAULT epoch_ms(now())
);

CREATE TABLE mutations (
    mutation_id UUID DEFAULT gen_random_uuid(),
    detected_at BIGINT NOT NULL,
    machine_id VARCHAR NOT NULL,
    risk_score DOUBLE NOT NULL,
    innate_score DOUBLE,
    adaptive_score DOUBLE,
    dimensions VARCHAR[], -- ['relational', 'volumetric', 'temporal']
    status VARCHAR DEFAULT 'active',
    memory_match_id UUID,
    response_actions VARCHAR[],
    resolved_at BIGINT,

```

```
    resolution_notes VARCHAR
);

CREATE TABLE memory_cells (
    cell_id UUID DEFAULT gen_random_uuid(),
    created_at BIGINT NOT NULL,
    mutation_dimensions VARCHAR[],
    deviation_profile JSON,
    progression_steps JSON,
    effective_response VARCHAR[],
    times_matched INTEGER DEFAULT 0,
    true_matches INTEGER DEFAULT 0,
    false_matches INTEGER DEFAULT 0,
    effectiveness DOUBLE DEFAULT 0.5,
    source VARCHAR DEFAULT 'local'
);
```

17.3 Schéma SQLite (configuration et profils)

```

CREATE TABLE machines (
  machine_id TEXT PRIMARY KEY,
  hostname TEXT NOT NULL,
  first_seen INTEGER NOT NULL,
  os TEXT,
  os_version TEXT,
  organ TEXT,
  role TEXT,
  profile_maturity REAL DEFAULT 0.0,
  last_updated INTEGER
);

CREATE TABLE machine_peers (
  machine_id TEXT NOT NULL,
  peer_ip TEXT NOT NULL,
  peer_hostname TEXT,
  ports TEXT,          -- JSON array
  protocols TEXT,      -- JSON array
  direction TEXT,      -- 'outgoing', 'incoming', 'both'
  avg_daily_volume INTEGER,
  avg_daily_connections REAL,
  first_seen INTEGER,
  last_seen INTEGER,
  confidence REAL,
  PRIMARY KEY (machine_id, peer_ip)
);

CREATE TABLE machine_temporal (
  machine_id TEXT PRIMARY KEY,
  active_hour_start INTEGER,
  active_hour_end INTEGER,
  active_days TEXT,     -- JSON array
  avg_hourly_volume TEXT, -- JSON array [24]
  avg_daily_connections REAL,
  avg_daily_volume INTEGER
);

CREATE TABLE tolerance_entries (
  entry_id TEXT PRIMARY KEY,
  pattern TEXT NOT NULL, -- JSON
  frequency TEXT,        -- 'daily', 'weekly', 'monthly', 'annual'
  last_seen INTEGER,
  expected_next INTEGER,
  confidence REAL,
  source TEXT            -- 'manual' or 'learned'
);

CREATE TABLE detection_rules (
  rule_id TEXT PRIMARY KEY,
  rule_type TEXT NOT NULL, -- 'innate' or 'adaptive'
  pattern TEXT NOT NULL,   -- JSON
  threshold REAL NOT NULL,
  true_positives INTEGER DEFAULT 0,
  false_positives INTEGER DEFAULT 0,

```



```

    true_negatives INTEGER DEFAULT 0,
    false_negatives INTEGER DEFAULT 0,
    effectiveness REAL DEFAULT 0.5,
    status TEXT DEFAULT 'active',
    last_evaluated INTEGER
);

CREATE TABLE contexts (
    context_id TEXT PRIMARY KEY,
    context_type TEXT NOT NULL,
    affected_machines TEXT,      -- JSON array
    start_time INTEGER NOT NULL,
    end_time INTEGER NOT NULL,
    tolerance_adjustments TEXT, -- JSON
    created_by TEXT
);

CREATE TABLE organs (
    organ_name TEXT PRIMARY KEY,
    machines TEXT NOT NULL,      -- JSON array
    internal_density REAL,
    external_peers TEXT          -- JSON array
);

```

17.4 Estimation des volumes

Pour un réseau de 100 machines :

Événements réseau	: ~500/machine/minute	= 50 000/min	= 72M/jour
Événements process	: ~10/machine/minute	= 1 000/min	= 1.4M/jour
Événements système	: ~1/machine/minute	= 100/min	= 144K/jour

Total : ~73M événements/jour

Taille estimée (compressé dans DuckDB) :

- Événement réseau moyen : ~100 octets compressé
- 72M * 100 = ~7 Go/jour
- Rétention 30 jours = ~210 Go

Serveur Core recommandé :

- CPU : 4 cores minimum
- RAM : 16 Go minimum
- Disque : 500 Go SSD
- Réseau : 1 Gbps

17.5 Sécurité de Thymus lui-même

Authentification :

- TLS mutuel entre Sensors et Core (certificats auto-signés par le Core)
- Chaque Sensor possède un certificat unique émis lors de l'enregistrement
- API Dashboard : authentification par token

Chiffrement :

- Toutes les communications gRPC : TLS 1.3
- Base DuckDB : chiffrement au repos (optionnel)
- Backup : chiffré

Intégrité :

- Le Sensor vérifie l'intégrité de son propre binaire au démarrage
- Les mises à jour sont signées
- Les commandes du Core sont authentifiées (double signature)

Protection contre la compromission du Core :

- Si le Core est compromis, les Sensors continuent de fonctionner en mode autonome
- Les Sensors n'exécutent que des commandes prédéfinies (pas d'exécution arbitraire)
- Le Core ne peut pas accéder aux données des machines (il reçoit des événements, pas des fichiers)

FIN DU DOCUMENT

Ce document constitue la base de conception pour le développement de Thymus. Il sera mis à jour au fur et à mesure de l'avancement du projet.

Pour toute question technique, contacter l'équipe d'ingénierie Thymus.

Thymus - Protéger l'écosystème numérique.